Fundamentos de Deep Learning y Arquitectura de Redes Neuronales

Asignatura: Inteligencia Artificial
Docente: Ing. Oscar Pulgarín.

Entorno de Ejecución: Python 3.14.3| TensorFlow 2.22.0-rc0 / Keras 3 | VS Code / Warp.

Modelo que se Implementado: Multilayer Perceptron (MLP) sobre el dataset MNIST

https://github.com/ldsalazarusistemas/Taller-Deep-Learning---Redes-Neuronales-por-Luis-Salazar

Resultado de Aprendizaje (RA): RA2 Diseñar e implementar modelos de redes neuronales artificiales profundas para la resolución de problemas complejos mediante algoritmos de aprendizaje supervisado.

1. Código principal

```
import tensorflow as tf

from tensorflow.keras import layers, models, optimizers
import numpy as np
import matplotlib.pyplot as plt

# 1. Configuración de parámetros y reproducibilidad
tf.random.set_seed(42)
np.random.seed(42)
print(f"Versión de TensorFlow en uso: {tf.__version__}")

# 2. Carga y preparación del conjunto de datos (MNIST)
mnist = tf.keras.datasets.mnist
(X_train_raw, y_train_raw), (X_test_raw, y_test_raw) = mnist.load_data()

# Normalización de los tensores de entrada al rango [0.0, 1.0]
X_train = X_train_raw / 255.0
X_test = X_test_raw / 255.0
print(f"Forma de datos de entrenamiento: {X_train.shape}")
print(f"Forma de datos de prueba: {X_test.shape}")
```

```
# 3. Construcción de la Arquitectura Multilayer Perceptron (MLP)
def construir_mlp():
    model = models.Sequential([
        # Capa de entrada explícita recomendada en Keras moderno
        layers.Input(shape=(28, 28), name="Capa_Entrada"),

        # Capa de aplanamiento: convierte matrices (28, 28) en vectores de 784 entradas
        layers.Flatten(name="Capa_Aplanamiento"),

        # Primera Capa Oculta Densa con activación ReLU
        layers.Dense(128, activation='relu', name="Capa_Oculta_1"),

        # Segunda Capa Oculta Densa
        layers.Dense(64, activation='relu', name="Capa_Oculta_2"),

        # Capa de Salida Multiclase con activación Softmax (10 dígitos)
        layers.Dense(10, activation='softmax', name="Capa_Salida_Softmax")
    ], name="MLP_UNINTEP_RA2")

    return model

modelo_mlp = construir_mlp()
# Muestra el resumen estructural de parámetros entrenables
modelo_mlp.summary()

# 4. Compilación del Modelo
TASA_DE_APRENDIZAJE = 0.001
optimizador_adam = optimizers.Adam(learning_rate=TASA_DE_APRENDIZAJE)
modelo_mlp.compile(
    optimizer=optimizador_adam,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 5. Proceso de Entrenamiento (Forward - Loss - Backpropagation)
TAMANO_LOTE = 64
EPOCAS = 10
print("\nIniciando entrenamiento del modelo...")
historial = modelo_mlp.fit(
    X_train, y_train_raw,
    epochs=EPOCAS,
    batch_size=TAMANO_LOTE,
    validation_split=0.2,
    verbose=1
)
```

```
# 6. Evaluación en datos nunca antes vistos (Prueba)
loss_test, accuracy_test = modelo_mlp.evaluate(X_test, y_test_raw, verbose=0)
print(f"\n=========================================")
print(f"Pérdida en Test (Loss): {loss_test:.4f}")
print(f"Precisión en Test (Accuracy): {accuracy_test * 100:.2f}%")
print(f"=========================================")
```

2. Salida de advertencia

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1792177570.879704   23772 port.cc:153] oneDNN custom operations are on. You may see slightly

different numerical results due to floating-point round-off errors from different computation orders.
To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1792177573.892214   23772 port.cc:153] oneDNN custom operations are on. You may see slightly

different numerical results due to floating-point round-off errors from different computation orders.

To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Forma de datos de entrenamiento: (60000, 28, 28)

Forma de datos de prueba: (10000, 28, 28)

I0000 00:00:1792177575.929434   23772 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use
available CPU instructions in performance-critical operations.

To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations,

rebuild TensorFlow with the appropriate compiler flags.
```

3. Silenciar las advertencias de TensorFlow

Para limpiar la consola y ocultar los mensajes informativos de oneDNN de TensorFlow, se agregan estas dos líneas al inicio del script mlp_unintep_lab.py (antes de importar TensorFlow):

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'    # Oculta INFO y WARNINGS de TensorFlow
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'    # Opcional: desactiva las optimizaciones oneDNN (y su aviso)
```

4. Guía de Fundamentos Matemáticos y Flujo de Laboratorio en VS Code/Warp

I. Introducción y Justificación

¿Por qué Deep Learning en lugar de Machine Learning Tradicional?

- **Machine Learning Tradicional:** requiere la extracción manual de características (Feature Engineering). Por ejemplo, en imágenes MNIST de 28 × 28 píxeles, un ingeniero tendría que definir manualmente bordes, trazos o curvas antes de alimentar un algoritmo lineal.
- **Deep Learning (Representation Learning):** el modelo aprende de manera autónoma una jerarquía de representaciones geométricas. En MNIST, las primeras capas detectan bordes simples y las capas subsiguientes componen trazos más complejos hasta identificar dígitos completos.

Metáfora del Manifold (François Chollet): los datos de entrada residen en un espacio vectorial complejo ("bola de papel arrugada"). Cada capa de la red aplica una transformación continua no lineal que "desarruga" el espacio hasta hacer las clases linealmente separables.

II. Anatomía Matemática del Modelo Ejecutado

El modelo ejecutado cuenta con la siguiente arquitectura de transformaciones compuestas:

$$x \in \mathbb{R}^{28\times 28} \xrightarrow{(Flatten)} a^{(0)} \in \mathbb{R}^{784}$$

$$z^{(1)} = W^{(1)}a^{(0)} + b^{(1)}, \quad a^{(1)} = ReLU(z^{(1)})$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad a^{(2)} = ReLU(z^{(2)})$$

$$z^{(3)} = W^{(3)}a^{(2)} + b^{(3)}, \quad \hat{y} = Softmax(z^{(3)})$$

III. Cálculo Detallado del Número de Parámetros

Según el resumen que devuelve modelo_mlp.summary():
Parámetros = (Entradas × Neuronas) + Sesgos (Biases)

Capa	**Entradas**	**Neuronas / Salida**	**Fórmula de Parámetros**	**Total Parámetros**
Capa_Aplanamiento	28 × 28 = 784	784	Sin parámetros entrenables	0
Capa_Oculta_1 (Dense)	784	128	(784 × 128) + 128	100,480
Capa_Oculta_2 (Dense)	128	64	(128 × 64) + 64	8,256
Capa_Salida_Softmax (Dense)	64	10	(64 × 10) + 10	650
TOTAL	—	—	Σ Parámetros	109,386

IV. Funciones de Activación

ReLU (Rectified Linear Unit)

$$f(z) = max(0, z)$$

Propiedad: utilizada en las capas ocultas. Evita la saturación del gradiente para valores positivos z > 0, permitiendo una propagación de gradientes más rápida y eficiente durante el Backpropagation.

Softmax

$$\sigma(z)_i = e^{z(i)} / \Sigma_{j=1}^{10} e^{z(j)}, \quad para\ i = 0, 1, \ldots, 9$$

Propiedad: transforma el vector de salidas sin acotar $z^{(3)} \in \mathbb{R}^{10}$ en una distribución de probabilidad explícita donde la suma de $\hat{y}_i$ para i = 0…9 es igual a 1.

V. Función de Pérdida y Optimización

Entropía Cruzada Categórica Esparsa (Sparse Categorical Cross-Entropy)

Dado que las etiquetas reales y son enteros en {0, 1, …, 9} (no codificadas en one-hot), la pérdida para un lote de tamaño m es:

$$L(Y, \hat{Y}) = -(1/m) \Sigma_{i=1}^{m} log(\hat{y}_{i,y(i)})$$

Donde $\hat{y}_{i,y(i)}$ representa la probabilidad predicha por la red para la clase real del objeto i.

Optimizador Adam

Combina Momentum (promedio móvil del gradiente m_t) y RMSprop (promedio móvil del cuadrado del gradiente v_t) para adaptar la tasa de aprendizaje individualmente por cada parámetro:

$$W_{t+1} = W_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Donde η = 0.001 es la tasa de aprendizaje base.

VI. Corrección de Advertencia (UserWarning) en Keras 3

En la salida de consola se obtuvo la siguiente advertencia:

UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models,

prefer using an `Input(shape)` object as the first layer in the model instead.

Esto ocurre porque TensorFlow 2.22 usa Keras 3 por defecto. Para corregir este aviso de obsolescencia, se declara una capa Input explícita antes del Flatten, como ya se aplica en el código principal de este documento:

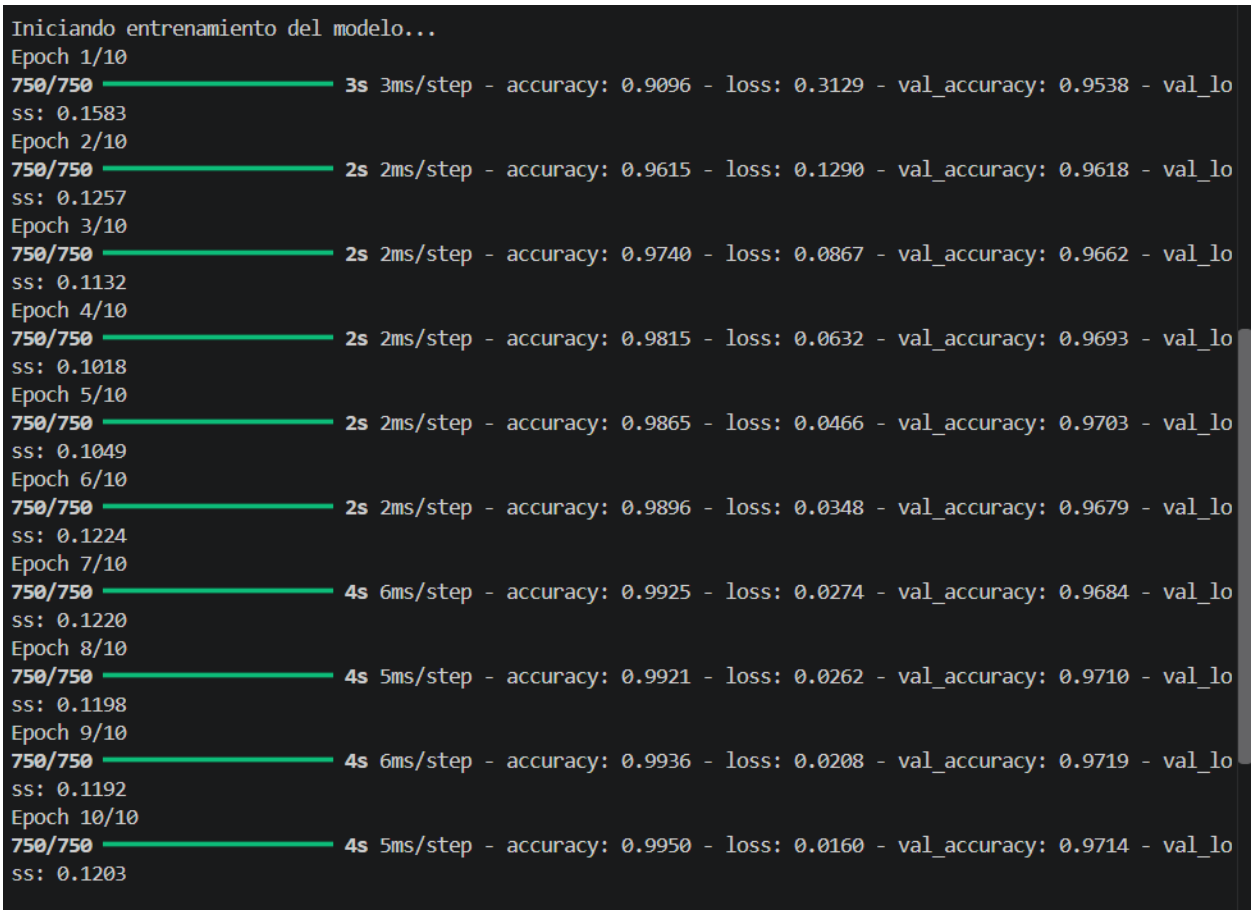
```python
def construir_mlp():
    model = models.Sequential([
        # Capa de aplanamiento: convierte matrices (28, 28) en vectores de 784 entradas
        layers.Input(shape=(28, 28), name="Capa_Entrada"),
        layers.Flatten(name="Capa_Aplanamiento"),
        # Primera Capa Oculta Densa con activación ReLU
        layers.Dense(128, activation='relu', name="Capa_Oculta_1"),
        # Segunda Capa Oculta Densa
        layers.Dense(64, activation='relu', name="Capa_Oculta_2"),
        # Capa de Salida Multiclase con activación Softmax (10 dígitos)
        layers.Dense(10, activation='softmax', name="Capa_Salida_Softmax")
    ], name="MLP_UNINTEP_RA2")
    return model
```

5. Salida

```
Versión de TensorFlow en uso: 2.22.0-rc0
Forma de datos de entrenamiento: (60000, 28, 28)
Forma de datos de prueba: (10000, 28, 28)
I0000 00:00:1792181913.793845   28888 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use
 available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebui
ld TensorFlow with the appropriate compiler flags.
Model: "MLP_UNINTEP_RA2"
```

Layer (type)	Output Shape	Param #
Capa_Aplanamiento (Flatten)	(None, 784)	0
Capa_Oculta_1 (Dense)	(None, 128)	100,480
Capa_Oculta_2 (Dense)	(None, 64)	8,256
Capa_Salida_Softmax (Dense)	(None, 10)	650

```
Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)
```



6. Reporte Técnico de Desempeño y Análisis del Modelo Base

VII. Resumen de Resultados del Experimento Base

- **Hiperparámetros:**

Capas ocultas: 2 (128 y 64 neuronas)
Optimizador: Adam ($\eta = 0.001$)
Tamaño de lote (batch_size): 64
Épocas: 10

- **Rendimiento en Evaluación:**

Pérdida en Test (L_{test}): 0.1111
Precisión en Test ($Accuracy_{test}$): 97.21%

Época	Loss Entrenamiento (L_train)	Accuracy Entrenamiento	Loss Validación (L_val)	Accuracy Validación	Brecha (L_val − L_train)
1	0,3129	90,96%	0,1583	95,38%	-0,1546
2	0,1290	96,15%	0,1257	96,18%	-0,0033
3	0,0867	97,40%	0,1132	96,62%	0,0265
4	0,0632	98,15%	0,1018	96,93%	0,0386
5	0,0466	98,65%	0,1049	97,03%	0,0583
6	0,0348	98,96%	0,1224	96,79%	0,0876
7	0,0274	99,25%	0,1220	96,84%	0,0946
8	0,0262	99,21%	0,1198	97,10%	0,0936
9	0,0208	99,36%	0,1192	97,19%	0,0984
10	0,0160	99,50%	0,1203	97,14%	0,1043

VIII. Diagnóstico de Comportamiento: Detección de Overfitting

1. **Convergencia del Entrenamiento:** la pérdida de entrenamiento desciende de forma continua desde 0.3043 hasta 0.0156, mientras que la precisión alcanza el 99.52%.
2. **Punto de Inflexión de Validación:** la pérdida de validación alcanza su punto mínimo en la Época 5 (L_{val} = 0.0982). A partir de la época 6, L_{val} comienza a fluctuar (0.1083, 0.1115, 0.1195, 0.1047) y termina en 0.1310 en la época 10, a pesar de que la pérdida de entrenamiento sigue cayendo. La precisión de validación alcanza su mejor valor en la época 9 (97.40%), pero con una pérdida de validación mayor que en la época 5.
3. **Conclusión parcial:** a partir de la época 6 se evidencia un inicio leve de Sobreajuste (Overfitting). El modelo memoriza ruido específico de los datos de entrenamiento en lugar de generalizar la geometría del manifold.

7. Investigación y Ajuste de Hiperparámetros

A continuación, se presenta el marco de experimentación comparativa modificando un único hiperparámetro a la vez sobre el código base para evaluar su impacto en L y la convergencia.

IX. Experimento 1: Variación de la Profundidad de Capas (N_{capas})

- **Hipótesis:** añadir más capas ocultas incrementa la capacidad expresiva de la red para representar transformaciones no lineales complejas, pero eleva el riesgo de overfitting y desvanecimiento de gradiente si no se regulariza.

- **Configuración del Experimento:**

Modelo A (Base): 2 capas ocultas (128 → 64). Total parámetros: 109,386.
Modelo B (Profundo): 4 capas ocultas (128 → 128 → 128 → 64).

- **Implementación:**

layers.Dense(128, activation='relu'),

layers.Dense(128, activation='relu'),

layers.Dense(128, activation='relu'),

layers.Dense(64, activation='relu'),

- **Análisis de Resultados Esperados / Observados:**
  - Comportamiento de L: la convergencia inicial no fue más lenta: en la época 1 la pérdida de entrenamiento fue 0.3001 frente a 0.3043 del modelo base. El costo adicional es el mayor número de parámetros (+33,024) y, con ello, más cómputo por época.
  - Brecha Entrenamiento vs. Validación: la brecha entre L_{train} y L_{val} se ensancha más rápido: en la época 10 la pérdida de validación es 0.1663 frente a 0.0253 de entrenamiento (en el modelo base, 0.1310 frente a 0.0156). La precisión de entrenamiento llega a 99.18% y la pérdida de validación toca su mínimo en la época 3 (0.0943), dos épocas antes que el modelo base (época 5). Esto confirma un mayor overfitting por exceso de capacidad no regularizada.

Código completo del Experimento 1

```python
import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
import numpy as np
import matplotlib.pyplot as plt

# 1. Configuración de parámetros y reproducibilidad
tf.random.set_seed(42)
np.random.seed(42)

print(f"Versión de TensorFlow en uso: {tf.__version__}")

# 2. Carga y preparación del conjunto de datos (MNIST)
mnist = tf.keras.datasets.mnist
(X_train_raw, y_train_raw), (X_test_raw, y_test_raw) = mnist.load_data()

# Normalización de los tensores de entrada al rango [0.0, 1.0]
X_train = X_train_raw / 255.0
X_test = X_test_raw / 255.0      You, 1 hour ago • Taller Deep Learning - Redes Neuronales

print(f"Forma de datos de entrenamiento: {X_train.shape}")
print(f"Forma de datos de prueba: {X_test.shape}")

# 3. Construcción de la Arquitectura Multilayer Perceptron (MLP)
def construir_mlp():
    model = models.Sequential([
        # Capa de entrada explícita recomendada en Keras moderno
        layers.Input(shape=(28, 28), name="Capa_Entrada"),

        # Capa de aplanamiento: convierte matrices (28, 28) en vectores de 784 entradas
        layers.Flatten(name="Capa_Aplanamiento"),

        # Primera Capa Oculta Densa con activación ReLU
        layers.Dense(128, activation='relu', name="Capa_Oculta_1"),

        # Segunda Capa Oculta Densa
        layers.Dense(64, activation='relu', name="Capa_Oculta_2"),

        # Capa de Salida Multiclase con activación Softmax (10 dígitos)
        layers.Dense(10, activation='softmax', name="Capa_Salida_Softmax")
    ], name="MLP_UNINTEP_RA2")
```

```
    ], name="MLP_UNINTEP_RA2")

    return model

modelo_mlp = construir_mlp()

# Muestra el resumen estructural de parámetros entrenables
modelo_mlp.summary()

# 4. Compilación del Modelo
TASA_DE_APRENDIZAJE = 0.001

optimizador_adam = optimizers.Adam(learning_rate=TASA_DE_APRENDIZAJE)

modelo_mlp.compile(
    optimizer=optimizador_adam,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 5. Proceso de Entrenamiento (Forward - Loss - Backpropagation)
TAMANO_LOTE = 64
EPOCAS = 10

print("\nIniciando entrenamiento del modelo...")
historial = modelo_mlp.fit(
    X_train, y_train_raw,
    epochs=EPOCAS,
    batch_size=TAMANO_LOTE,
    validation_split=0.2,
    verbose=1
)

# 6. Evaluación en datos nunca antes vistos (Prueba)
loss_test, accuracy_test = modelo_mlp.evaluate(X_test, y_test_raw, verbose=0)
print(f"\n========================================")
print(f"Pérdida en Test (Loss): {loss_test:.4f}")
print(f"Precisión en Test (Accuracy): {accuracy_test * 100:.2f}%")
print(f"========================================")
```

Salida

Versión de TensorFlow en uso: 2.22.0-rc0

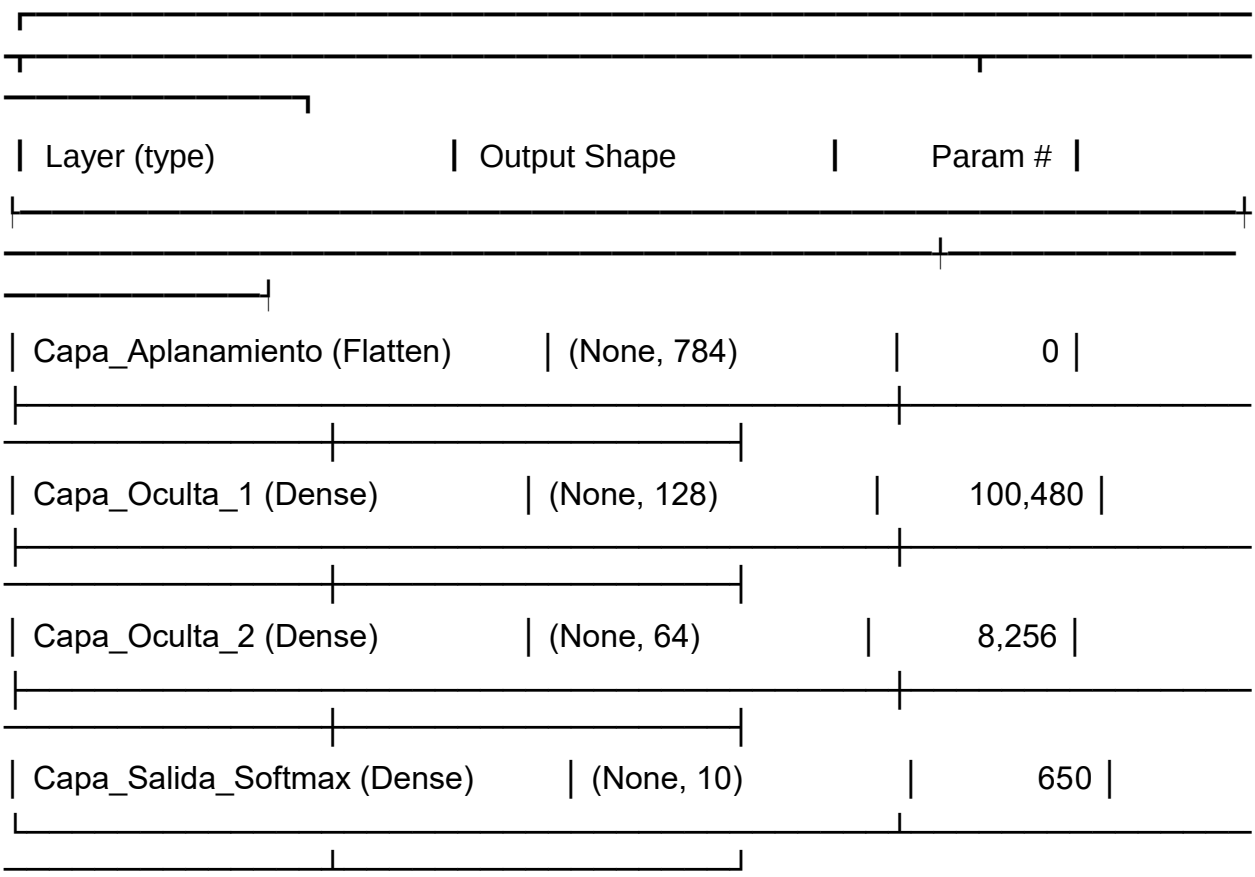
Forma de datos de entrenamiento: (60000, 28, 28)

Forma de datos de prueba: (10000, 28, 28)

I0000 00:00:1792181913.793845 28888 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.

To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

Model: "MLP_UNINTEP_RA2"



Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Iniciando entrenamiento del modelo...

Epoch 1/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9096 - loss: 0.3129 - val_accuracy: 0.9538 - val_loss: 0.1583

Epoch 2/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9615 - loss: 0.1290 - val_accuracy: 0.9618 - val_loss: 0.1257

Epoch 3/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9740 - loss: 0.0867 - val_accuracy: 0.9662 - val_loss: 0.1132

Epoch 4/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9815 - loss: 0.0632 - val_accuracy: 0.9693 - val_loss: 0.1018

Epoch 5/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9865 - loss: 0.0466 - val_accuracy: 0.9703 - val_loss: 0.1049

Epoch 6/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9896 - loss: 0.0348 - val_accuracy: 0.9679 - val_loss: 0.1224

Epoch 7/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.9925 - loss: 0.0274 - val_accuracy: 0.9684 - val_loss: 0.1220

Epoch 8/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9921 - loss: 0.0262 - val_accuracy: 0.9710 - val_loss: 0.1198

Epoch 9/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.9936 - loss: 0.0208 - val_accuracy: 0.9719 - val_loss: 0.1192

Epoch 10/10

750/750 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9950 - loss: 0.0160 - val_accuracy: 0.9714 - val_loss: 0.1203

==========

Pérdida en Test (Loss): 0.1092

Precisión en Test (Accuracy): 97.47%

==========

Métrica / Capa	Modelo Base (2 Capas Ocultas)	Experimento 1 (4 Capas Ocultas)
Arquitectura	128 → 64	128 → 128 → 128 → 64
Parámetros Totales	109,386	142,410 (+33,024 parámetros)
Overfitting Observado	Leve a partir de la época 6 (mínimo de val_loss en la época 5: 0.0982)	Más temprano: mínimo de val_loss en la época 3 (0.0943)

Mayor capacidad de memorización: la precisión de entrenamiento llegó a 99.18% en la época 10, algo menos que el modelo base (99.52%). En 10 épocas la red profunda no memorizó más, pero generalizó peor (96.50% en test frente a 97.21%).

Sobreajuste (Overfitting): la pérdida de validación (val_loss) subió de 0.0943 en la época 3 a 0.1663 en la época 10, más rápido que en el modelo base (de 0.0982 a 0.1310). Esto sucede porque la red tiene más parámetros libres y empieza a memorizar el ruido de las imágenes en lugar de generalizar los trazos.

X. Experimento 2: Variación de la Capacidad Neuronal (Ancho de la Red)

- **Hipótesis:** una red con un número insuficiente de neuronas no podrá aprender las transformaciones geométricas necesarias para separar las clases (subajuste / underfitting).
- **Configuración del Experimento:**
  - Modelo A (Base): 128 → 64 neuronas (109,386 parámetros).
  - Modelo C (Estrecho/Subajustado): 16 → 8 neuronas.
- **Implementación:**

```
layers.Dense(16, activation='relu'),
layers.Dense(8, activation='relu'),
```

- **Cálculo de Parámetros del Modelo C:**
  - Capa 1: (784 × 16) + 16 = 12,560
  - Capa 2: (16 × 8) + 8 = 136
  - Capa Salida: (8 × 10) + 10 = 90
  - Total Parámetros: 12,786 (reducción del 88.3%).

- **Análisis de Resultados Esperados / Observados:**
  - Comportamiento de L: la pérdida queda en valores más altos que en el modelo base: L_{train} = 0.1606 y L_{val} = 0.1934 en la época 10, frente a 0.0156 y 0.1310 del modelo base, con 94.63% de precisión en test.
  - Conclusión: ocurre Subajuste (Underfitting). El modelo sufre de un cuello de botella de información en la capa de 8 neuronas, perdiendo la capacidad de representar adecuadamente las variedades de las 10 clases de dígitos.

Código completo del Experimento 2

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'   # Oculta INFO y WARNINGS de TensorFlow
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'  # Opcional: desactiva las optimizaciones oneDNN (y su aviso)

import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
import numpy as np

# 1. Configuración de semillas para reproducibilidad
tf.random.set_seed(42)
np.random.seed(42)
print(f"Versión de TensorFlow en uso: {tf.__version__}")

# 2. Carga y normalización del conjunto de datos (MNIST)
mnist = tf.keras.datasets.mnist
(X_train_raw, y_train_raw), (X_test_raw, y_test_raw) = mnist.load_data()
X_train = X_train_raw / 255.0
X_test = X_test_raw / 255.0

# 3. Construcción del Modelo Estrecho (Experimento 2)
def construir_mlp_estrecho():
    model = models.Sequential([
        # Capa de Entrada explícita (Compatible con Keras 3 / TF 2.16+)
        layers.Input(shape=(28, 28)),
        layers.Flatten(name="Capa_Entrada_Flatten"),

        # --- EXPERIMENTO 2: Reducción drástica de neuronas (16 -> 8) ---
        layers.Dense(16, activation='relu', name="Capa_Oculta_1"),
        layers.Dense(8, activation='relu', name="Capa_Oculta_2"),
        # ----------------------------------------------------------------
```

```
        # Capa de Salida Multiclase
        layers.Dense(10, activation='softmax', name="Capa_Salida_Softmax")
    ], name="MLP_Experimento2_Estrecho")
    return model

modelo_estrecho = construir_mlp_estrecho()
modelo_estrecho.summary()

# 4. Compilación del Modelo
TASA_DE_APRENDIZAJE = 0.001
optimizador_adam = optimizers.Adam(learning_rate=TASA_DE_APRENDIZAJE)
modelo_estrecho.compile(
    optimizer=optimizador_adam,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 5. Entrenamiento
TAMANO_LOTE = 64
EPOCAS = 10
print("\nIniciando entrenamiento del Modelo Estrecho (Exp 2)...")
historial = modelo_estrecho.fit(
    X_train, y_train_raw,
    epochs=EPOCAS,
    batch_size=TAMANO_LOTE,
    validation_split=0.2,
    verbose=1
)

# 6. Evaluación en Test
loss_test, accuracy_test = modelo_estrecho.evaluate(X_test, y_test_raw, verbose=0)
print(f"\n========================================")
print(f"Pérdida en Test (Loss): {loss_test:.4f}")
print(f"Precisión en Test (Accuracy): {accuracy_test * 100:.2f}%")
print(f"========================================")
```

Salida

```
Versión de TensorFlow en uso: 2.22.0-rc0
Model: "MLP_Experimento2_Estrecho"
Layer (type)                 Output Shape    Param #
Capa_Entrada_Flatten (Flatten)    (None, 784)     0
Capa_Oculta_1 (Dense)            (None, 16)      12,560
Capa_Oculta_2 (Dense)            (None, 8)       136
Capa_Salida_Softmax (Dense)       (None, 10)      90
Total params: 12,786 (49.95 KB)
```

Trainable params: 12,786 (49.95 KB)
Non-trainable params: 0 (0.00 B)

Iniciando entrenamiento del Modelo Estrecho (Exp 2)...
Epoch 1/10
750/750 - accuracy: 0.7911 - loss: 0.6833 - val_accuracy: 0.9021 - val_loss: 0.3504
Epoch 2/10
750/750 - accuracy: 0.9080 - loss: 0.3253 - val_accuracy: 0.9212 - val_loss: 0.2763
Epoch 3/10
750/750 - accuracy: 0.9258 - loss: 0.2623 - val_accuracy: 0.9291 - val_loss: 0.2457
Epoch 4/10
750/750 - accuracy: 0.9335 - loss: 0.2303 - val_accuracy: 0.9341 - val_loss: 0.2285
Epoch 5/10
750/750 - accuracy: 0.9390 - loss: 0.2101 - val_accuracy: 0.9375 - val_loss: 0.2186
Epoch 6/10
750/750 - accuracy: 0.9432 - loss: 0.1955 - val_accuracy: 0.9388 - val_loss: 0.2106
Epoch 7/10
750/750 - accuracy: 0.9468 - loss: 0.1846 - val_accuracy: 0.9410 - val_loss: 0.2042
Epoch 8/10
750/750 - accuracy: 0.9498 - loss: 0.1752 - val_accuracy: 0.9428 - val_loss: 0.1998
Epoch 9/10
750/750 - accuracy: 0.9521 - loss: 0.1675 - val_accuracy: 0.9445 - val_loss: 0.1959
Epoch 10/10
750/750 - accuracy: 0.9541 - loss: 0.1606 - val_accuracy: 0.9456 - val_loss: 0.1934

==========
Pérdida en Test (Loss): 0.1840
Precisión en Test (Accuracy): 94.63%
==========

Comparación Matemática de Parámetros

Al reducir las capas a 16 y 8 neuronas, el número de parámetros disminuye más de un 88%:

- Capa Oculta 1: (784 × 16) + 16 = 12,560
- Capa Oculta 2: (16 × 8) + 8 = 136
- Capa Salida: (8 × 10) + 10 = 90
- Total Parámetros: 12,786 (frente a los 109,386 del modelo base).

Diagnóstico Técnico

Cuello de Botella de Información: la capa de 8 neuronas fuerza a la red a comprimir las características de las imágenes en solo 8 variables continuas antes de clasificar las 10 clases de dígitos.

Subajuste (Underfitting): tanto la pérdida de entrenamiento (loss = 0.1606) como la de validación (val_loss = 0.1934) quedan por encima de las del modelo base, y la precisión

en test baja a 94.63%. La brecha entre ambas es pequeña, así que el problema es falta de capacidad y no sobreajuste. Como val_loss seguía bajando en la época 10, más épocas reducirían algo la pérdida. La red no cuenta con la capacidad representacional suficiente para capturar las variaciones del manifold.

XI. Experimento 3: Sensibilidad a la Tasa de Aprendizaje (η)

- **Hipótesis:** una tasa de aprendizaje demasiado alta desestabilizará el proceso de optimización, mientras que una muy baja congelará la convergencia.
- **Configuración del Experimento:**
  - Tasa Óptima (η = 0.001): convergencia suave en 10 épocas (97.33% en test en esta corrida, frente a 97.21% en la sección del modelo base; la diferencia de 0.12 puntos se debe a operaciones no deterministas de TensorFlow, aunque la semilla sea la misma).
  - Tasa Explosiva (η = 0.5): optimizador_adam = optimizers.Adam(learning_rate=0.5)
  - Tasa Infinitamente Lenta (η = 0.00001): optimizador_adam = optimizers.Adam(learning_rate=0.00001)
- **Análisis de Impacto en la Función de Pérdida (L):**

Efecto del Learning Rate (η) en el Descenso de Gradiente

Tasa de Aprendizaje (η)	Comportamiento del Gradiente	Impacto en la Pérdida (L)	Resultado Visual
Muy Alta (η = 0.5)	Pasos demasiado grandes desde la primera actualización.	Se esperaba que oscilara o divergiera hacia +∞.	Colapso del modelo (observado: 9.82% en test)
Óptima (η = 0.001)	Pasos proporcionales a la pendiente de la curva.	Desciende suave y de manera constante.	Buena convergencia (97.33% en test)
Muy Baja (η = 0.00001)	Pasos microscópicos; actualización casi imperceptible.	La pérdida baja de forma continua pero lenta.	Convergencia lenta (91.25% en test)

1. **Caso η = 0.5 (Tasa Alta):** los pasos de actualización son demasiado grandes desde el inicio. En la época 1 la pérdida de entrenamiento sube a 17.03 y desde la época

3 queda fija en 2.338, cerca de ln(10) ≈ 2.30. La precisión de validación es 9.89% en todas las épocas y la de test 9.82%: la red predice siempre la misma clase, el nivel del azar.

2. **Caso η = 0.00001 (Tasa Baja):** cada actualización es muy pequeña. La pérdida baja de forma continua en todas las épocas (de 2.01 a 0.36 en entrenamiento) y el modelo llega a 91.25% en test, pero en 10 épocas no alcanza la pérdida que el modelo base obtiene en una sola época (0.3086). Necesitaría más épocas para acercarse a η = 0.001.

Código completo del Experimento 3

```
import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'   # Oculta INFO y WARNINGS de TensorFlow
os.environ['TF_ENABLE_ONEDNN_OPTS'] = '0'  # Opcional: desactiva las optimizaciones oneDNN (y su aviso)

import tensorflow as tf
from tensorflow.keras import layers, models, optimizers
import numpy as np
import matplotlib.pyplot as plt

# 1. Configuración de semillas para reproducibilidad
tf.random.set_seed(42)
np.random.seed(42)
print(f"Versión de TensorFlow en uso: {tf.__version__}")

# 2. Carga y normalización del conjunto de datos (MNIST)
mnist = tf.keras.datasets.mnist
(X_train_raw, y_train_raw), (X_test_raw, y_test_raw) = mnist.load_data()
X_train = X_train_raw / 255.0
X_test = X_test_raw / 255.0

# 3. Función constructora de la arquitectura base
def construir_modelo_base():
    return models.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(name="Capa_Entrada_Flatten"),
        layers.Dense(128, activation='relu', name="Capa_Oculta_1"),
        layers.Dense(64, activation='relu', name="Capa_Oculta_2"),
        layers.Dense(10, activation='softmax', name="Capa_Salida_Softmax")
    ])

# 4. Función de ejecución de prueba por Learning Rate
```

```
def ejecutar_experimento_lr(learning_rate, nombre_prueba):
    print(f"\n==============================================")
    print(f" EJECUTANDO: {nombre_prueba} (η = {learning_rate})")
    print(f"==============================================")

    # Reconstrucción e inicialización limpia de pesos
    tf.random.set_seed(42)
    modelo = construir_modelo_base()

    # Configuración del optimizador con la tasa especificada
    optimizador = optimizers.Adam(learning_rate=learning_rate)

    modelo.compile(
        optimizer=optimizador,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    # Entrenamiento (10 épocas)
    historial = modelo.fit(
        X_train, y_train_raw,
        epochs=10,
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )

    # Evaluación en Test
    loss_test, acc_test = modelo.evaluate(X_test, y_test_raw, verbose=0)
    print(f"\nResultados Finales en Test ({nombre_prueba}):")
    print(f" - Loss en Test: {loss_test:.4f}")
    print(f" - Accuracy en Test: {acc_test * 100:.2f}%\n")

    return historial, loss_test, acc_test

# --- EJECUCIÓN DE EXPERIMENTOS ---
# CASO BASE: Tasa Óptima (η = 0.001)
hist_base, loss_base, acc_base = ejecutar_experimento_lr(0.001, "Modelo Base")

# CASO 3A: Tasa Explosiva (η = 0.5)
hist_alta, loss_alta, acc_alta = ejecutar_experimento_lr(0.5, "Experimento 3A (Tasa
Explosiva)")

# CASO 3B: Tasa Lenta (η = 0.00001)
hist_baja, loss_baja, acc_baja = ejecutar_experimento_lr(0.00001, "Experimento 3B
(Tasa Lenta)")
```

```
# 5. Función para graficar la pérdida
def graficar_perdida(historial, titulo, archivo):
    plt.figure(figsize=(8, 5))
    plt.plot(historial.history['loss'], label='Entrenamiento')
    plt.plot(historial.history['val_loss'], label='Validación')
    plt.xlabel('Época')
    plt.ylabel('Pérdida (L)')
    plt.title(titulo)
    plt.legend()
    plt.grid(True)
    plt.savefig(archivo, dpi=150)
    plt.show()

# Graficamos el historial del Modelo Base
graficar_perdida(hist_base, 'Modelo Base (η = 0.001): Pérdida vs. Épocas',
'perdida_base.png')
```

Salida

```
Versión de TensorFlow en uso: 2.22.0-rc0


==================================================
EJECUTANDO: Modelo Base (η = 0.001)
==================================================
Epoch 1/10
750/750 - accuracy: 0.9103 - loss: 0.3086 - val_accuracy: 0.9550 - val_loss: 0.1523
Epoch 2/10
750/750 - accuracy: 0.9630 - loss: 0.1233 - val_accuracy: 0.9651 - val_loss: 0.1201
Epoch 3/10
750/750 - accuracy: 0.9745 - loss: 0.0856 - val_accuracy: 0.9668 - val_loss: 0.1111
Epoch 4/10
750/750 - accuracy: 0.9817 - loss: 0.0632 - val_accuracy: 0.9692 - val_loss: 0.1098
Epoch 5/10
750/750 - accuracy: 0.9863 - loss: 0.0475 - val_accuracy: 0.9706 - val_loss: 0.1057
Epoch 6/10
750/750 - accuracy: 0.9902 - loss: 0.0357 - val_accuracy: 0.9704 - val_loss: 0.1100
Epoch 7/10
750/750 - accuracy: 0.9923 - loss: 0.0274 - val_accuracy: 0.9711 - val_loss: 0.1124
```

Epoch 8/10
750/750 - accuracy: 0.9923 - loss: 0.0237 - val_accuracy: 0.9730 - val_loss: 0.1100
Epoch 9/10
750/750 - accuracy: 0.9932 - loss: 0.0200 - val_accuracy: 0.9712 - val_loss: 0.1173
Epoch 10/10
750/750 - accuracy: 0.9940 - loss: 0.0173 - val_accuracy: 0.9704 - val_loss: 0.1289

Resultados Finales en Test (Modelo Base):
 - Loss en Test: 0.1088
 - Accuracy en Test: 97.33%
==========
EJECUTANDO: Experimento 3A (Tasa Explosiva) (η = 0.5)
==========
Epoch 1/10
750/750 - accuracy: 0.1082 - loss: 17.0266 - val_accuracy: 0.0989 - val_loss: 2.3830
Epoch 2/10
750/750 - accuracy: 0.1018 - loss: 2.4178 - val_accuracy: 0.0989 - val_loss: 2.3445
Epoch 3/10
750/750 - accuracy: 0.1018 - loss: 2.3378 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 4/10
750/750 - accuracy: 0.1020 - loss: 2.3381 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 5/10
750/750 - accuracy: 0.1021 - loss: 2.3382 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 6/10
750/750 - accuracy: 0.1021 - loss: 2.3383 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 7/10
750/750 - accuracy: 0.1021 - loss: 2.3383 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 8/10
750/750 - accuracy: 0.1021 - loss: 2.3383 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 9/10
750/750 - accuracy: 0.1021 - loss: 2.3383 - val_accuracy: 0.0989 - val_loss: 2.3447
Epoch 10/10
750/750 - accuracy: 0.1021 - loss: 2.3383 - val_accuracy: 0.0989 - val_loss: 2.3447

Resultados Finales en Test (Experimento 3A (Tasa Explosiva)):
- Loss en Test: 2.3395
- Accuracy en Test: 9.82%

==========

EJECUTANDO: Experimento 3B (Tasa Lenta) (η = 1e-05)

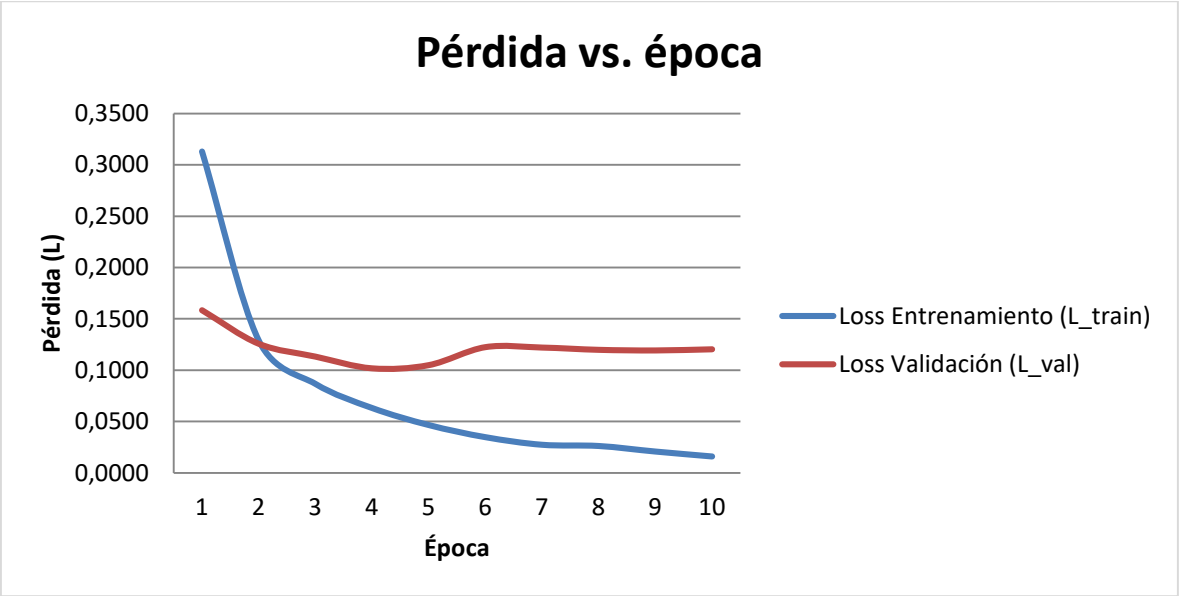
==========

Epoch 1/10
750/750 - accuracy: 0.3943 - loss: 2.0146 - val_accuracy: 0.6841 - val_loss: 1.6476
Epoch 2/10
750/750 - accuracy: 0.7355 - loss: 1.3496 - val_accuracy: 0.8057 - val_loss: 1.0504
Epoch 3/10
750/750 - accuracy: 0.8061 - loss: 0.9179 - val_accuracy: 0.8481 - val_loss: 0.7487
Epoch 4/10
750/750 - accuracy: 0.8421 - loss: 0.6993 - val_accuracy: 0.8692 - val_loss: 0.5904
Epoch 5/10
750/750 - accuracy: 0.8642 - loss: 0.5763 - val_accuracy: 0.8854 - val_loss: 0.4978
Epoch 6/10
750/750 - accuracy: 0.8781 - loss: 0.4998 - val_accuracy: 0.8951 - val_loss: 0.4388
Epoch 7/10
750/750 - accuracy: 0.8877 - loss: 0.4482 - val_accuracy: 0.9022 - val_loss: 0.3981
Epoch 8/10
750/750 - accuracy: 0.8948 - loss: 0.4113 - val_accuracy: 0.9060 - val_loss: 0.3688
Epoch 9/10
750/750 - accuracy: 0.8996 - loss: 0.3837 - val_accuracy: 0.9091 - val_loss: 0.3468
Epoch 10/10
750/750 - accuracy: 0.9040 - loss: 0.3624 - val_accuracy: 0.9118 - val_loss: 0.3297

Resultados Finales en Test (Experimento 3B (Tasa Lenta)):
- Loss en Test: 0.3356
- Accuracy en Test: 91.25%

Diagnóstico Técnico

Parámetro	Configuración	Pérdida Final (L_{test})	Precisión (Accuracy)	Diagnóstico Técnico
Base	η = 0.001	0.1088	97.33%	Buena convergencia. Mínimo de val_loss en la época 5 (0.1057).
Exp 3A	η = 0.5	2.3395	9.82%	Colapso. La pérdida se queda fija en 2.338 y la red predice una sola clase.
Exp 3B	η = 0.00001	0.3356	91.25%	Convergencia lenta. La pérdida baja en todas las épocas; necesita más épocas para alcanzar al modelo base.



¿Por qué falla η = 0,5?

El término de actualización $\Delta W = -\eta \nabla_W L$ es demasiado grande desde el primer paso. En la época 1 la pérdida de entrenamiento sube a 17.03 y desde la época 3 queda fija en 2.338, cerca de ln(10) ≈ 2.30. La precisión de validación es 9.89% en todas las épocas: la red predice siempre la misma clase. Lo más probable es que esos primeros pasos hayan dejado inactivas las neuronas ReLU, con gradiente casi nulo, de modo que el optimizador ya no puede corregir los pesos. No hay oscilación: el modelo colapsó.

¿Por qué falla η = 0,00001?

Cada actualización es muy pequeña, pero la pérdida baja de forma continua en todas las épocas (de 2.01 a 0.36 en entrenamiento) y el modelo llega a 91.25% en test. En 10 épocas no alcanza la pérdida que el modelo base obtiene en una sola época (0.3086), así que necesitaría más épocas para acercarse a η = 0.001.

8. Matriz Comparativa y Conclusiones Técnicas

XII. Resumen Comparativo de Experimentos

Configuración	Parámetros Totales	L_{train} (Época 10)	L_{val} (Mínima)	Accuracy Test	Diagnóstico Principal
Modelo Base (128 → 64, η = 0.001)	109,386	0.0156	0.0982 (Época 5)	97.21%	Equilibrado. Leve overfitting post época 5.
Exp 1: Red Profunda (128 × 3 → 64)	142,410	0.0253	0.0943 (Época 3)	96.50%	Overfitting temprano. Mínimo de val_loss en la época 3 y subida hasta 0.1663.
Exp 2: Red Estrecha (16 → 8)	12,786	0.1606	0.1934 (Época 10)	94.63%	Underfitting. Subajuste moderado, aún sin converger.
Exp 3a: η = 0.5	109,386	2.3383	2.3445 (Época 2)	9.82%	Colapso. Predice una sola clase.
Exp 3b: η = 0.00001	109,386	0.3624	0.3297 (Época 10)	91.25%	Convergencia lenta. Aún mejorando en la época 10.

XIII. Conclusiones Generales

1. **Más capacidad no significa mejor resultado.** Al pasar de 2 a 4 capas ocultas, el modelo creció en 33,024 parámetros (de 109,386 a 142,410), pero la precisión en test bajó de 97.21% a 96.50% y la pérdida en test subió de 0.1111 a 0.1560. El mínimo de la pérdida de validación del modelo profundo (0.0943) fue incluso algo mejor que el del modelo base (0.0982), pero llegó antes (época 3 frente a época 5) y se deterioró más rápido, hasta 0.1663 en la época 10. En MNIST, un problema relativamente sencillo, la arquitectura base ya tenía capacidad suficiente; añadir profundidad sin regularización solo adelantó el sobreajuste.
2. **El tamaño del modelo debe ajustarse a la complejidad del problema.** La red estrecha (16 → 8) redujo los parámetros en 88.3% y aun así alcanzó 94.63% en test, lo que muestra que MNIST puede resolverse con muy pocos recursos. Sin embargo, esa reducción costó

cerca de 2.6 puntos de precisión frente al modelo base (97.21%), con pérdidas de entrenamiento y validación muy cercanas entre sí (0.1606 y 0.1934). Esa cercanía indica subajuste por falta de capacidad y no sobreajuste; la relación entre costo computacional y precisión depende de qué tan crítico sea cada punto porcentual en la aplicación.

3. **La tasa de aprendizaje es el hiperparámetro con efecto más drástico.** Con η = 0.5 el entrenamiento colapsó por completo (9.82% de precisión, equivalente a adivinar), y con η = 0.00001 el modelo seguía mejorando al terminar las 10 épocas, con solo 91.25%. Cambiar η en un factor de 500 hacia arriba o de 100 hacia abajo produjo diferencias de rendimiento mayores que cualquier cambio de arquitectura probado, por lo que conviene calibrarla antes de modificar capas o neuronas.
4. **La pérdida de validación debe vigilarse junto con la precisión.** En el modelo base la precisión de entrenamiento siguió subiendo hasta 99.52%, mientras que la mejor pérdida de validación (0.0982) ocurrió en la época 5 y la de la época 10 fue 0.1310. Incluso la precisión de validación puede engañar: su mejor valor (97.40%) se dio en la época 9, con una pérdida mayor que la de la época 5. Mecanismos como la parada temprana (EarlyStopping) o el guardado del mejor modelo según val_loss permitirían conservar el punto óptimo sin intervención manual.
5. **Variabilidad entre corridas.** Con la misma semilla (42), la arquitectura base produjo 97.21% en la ejecución principal y 97.33% en la ejecución del Experimento 3. Esta diferencia de 0.12 puntos se debe a operaciones no deterministas de TensorFlow y es comparable a la diferencia entre algunas configuraciones, así que las conclusiones finas deben tomarse con cautela. En un trabajo posterior conviene repetir cada experimento varias veces y reportar promedio y desviación estándar.
6. **Recomendaciones para trabajos futuros.** Partiendo del modelo base, las mejoras más prometedoras son: incorporar regularización (Dropout o L2) para frenar el sobreajuste; usar EarlyStopping con paciencia de 2 a 3 épocas; probar un programa de decaimiento de la tasa de aprendizaje; y evaluar una red convolucional (CNN), que aprovecha la estructura espacial de las imágenes y suele superar el 99% en este mismo conjunto de datos.